

SDEV2171

Lesson 21: Persisting User Sessions

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Lesson Objectives

By the end of this lesson, students should be able to:

- explain why auth state disappears when it only lives in memory
- store and retrieve an auth session with secure local storage
- restore auth state when the app starts again
- validate a restored session with `GET /auth/me`
- clear invalid or signed-out session data safely

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson Roadmap

1. Revisit lesson 20’s working auth flow
2. Define the restart problem
3. Add secure storage helpers
4. Restore auth state during startup
5. Validate or clear the restored session
6. Checkpoint and lesson-22 handoff

WE ARE ESSENTIAL TO ALBERTA | NAIT

Starting Point From Lesson 20

- keep the same auth backend
- keep the same public and protected route groups
- keep the same protected study-step feature

What changes today

- auth state survives restart
- startup now has a restore phase
- invalid stored sessions get cleared cleanly

WE ARE ESSENTIAL TO ALBERTA | NAIT

Preferred Runtime For This Lesson

- use the development-build path for this lesson
- `expo-secure-store` belongs in the real app runtime
- restart testing and DevTools inspection are both part of today's workflow
- Expo Go is still useful as a comparison path, but not the main class path here

WE ARE ESSENTIAL TO ALBERTA | NAIT

Problem Statement

Lesson 20 works in memory only.

What that means

- log in
- close the app
- open it again
- the token is gone
- the app starts as signed out again

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why This Matters

- users expect to stay signed in between launches
- protected screens should not require a full login every time
- session restore is part of user experience, not just a backend feature

Still true

- sign-out must clear access immediately

WE ARE ESSENTIAL TO ALBERTA | NAIT

In Memory Versus Persisted

In-memory auth state

- fast
- simple
- lost on restart

Persisted auth session

- survives restart
- needs safe storage
- needs startup validation logic

WE ARE ESSENTIAL TO ALBERTA | NAIT

Storage Rule

Do not treat tokens like ordinary UI values.

Good fit for secure storage

- auth token
- saved auth session object

Not the point of secure storage today

- selected study steps
- form draft text
- one-screen loading state

WE ARE ESSENTIAL TO ALBERTA | NAIT

Session Shape

Today’s stored value can stay small:

```
{
  token: '...',
  user: {
    id: '...',
    name: '...',
    email: '...'
  }
}
```

Why store both

- token restores access
- user restores a stable signed-in identity immediately

WE ARE ESSENTIAL TO ALBERTA | NAIT

One Storage Helper File

Create one small helper that can:

- save the session
- read the session
- clear the session

Why

- keeps storage details out of screen code
- keeps the auth provider easier to read

WE ARE ESSENTIAL TO ALBERTA | NAIT

Web Fallback Note

- mobile path: `expo-secure-store`
- web fallback: local storage only if the lesson app is opened on web

Important distinction

- the class security message still centers on the mobile secure-storage path

WE ARE ESSENTIAL TO ALBERTA | NAIT

New Provider Responsibilities

The auth provider now must:

- store live auth state
- save session data after login or registration
- restore saved session on startup
- validate restored data with `/auth/me`
- clear invalid stored data

WE ARE ESSENTIAL TO ALBERTA | NAIT

New Startup State

Add one more provider flag:

```
const [isRestoring, setIsRestoring] = useState(true);
```

Why

- the app needs a short restore phase before route redirects run

WE ARE ESSENTIAL TO ALBERTA | NAIT

Restore Flow

1. app starts
2. provider reads stored session
3. if none exists, finish restore and stay public
4. if one exists, validate it with `/auth/me`
5. keep it or clear it

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why Validate With `/auth/me`

- stored data might be stale
- token could be expired or invalid
- backend should remain the source of truth for access

Class rule

- `/auth/me` is the single preferred validation proof for restored sessions

WE ARE ESSENTIAL TO ALBERTA | NAIT

When To Clear Stored Data

- login token is invalid
- stored session object is malformed
- user chooses logout

Result

- clear storage
- clear live auth state
- return to the public route flow

WE ARE ESSENTIAL TO ALBERTA | NAIT

Route-Gating Change

Lesson 20 route logic was:

- authenticated -> protected
- signed out -> public

Lesson 21 adds one step first:

- restoring -> wait

WE ARE ESSENTIAL TO ALBERTA | NAIT

Wait Before Redirecting

- entry route should not redirect before restore finishes
- protected layout should not kick the user out before restore finishes

Otherwise

- you create a false sign-out flicker during app startup

WE ARE ESSENTIAL TO ALBERTA | NAIT

Recommended Build Order

1. add `expo-secure-store`
2. create the storage helper
3. save session data after login and registration
4. clear session data on logout
5. add restore logic in the provider
6. update route gates to respect `isRestoring`
7. restart-test the app

WE ARE ESSENTIAL TO ALBERTA | NAIT

Visible Clues To Confirm Success

1. sign in once
2. reach the protected route group
3. restart the app
4. land back in the protected route group without signing in again
5. log out
6. restart again and stay public

WE ARE ESSENTIAL TO ALBERTA | NAIT

Common Wrong Turns

- storing only the token but not the user shape expected by the app
- forgetting to save after registration as well as login
- running redirects before restore finishes
- restoring local data but never validating it with `/auth/me`
- clearing live state on logout but forgetting to clear storage

WE ARE ESSENTIAL TO ALBERTA | NAIT

Midpoint Check

By the middle of class, students should have:

- one storage helper file
- one saved session after login
- one clear place where logout clears storage
- one plan for the startup restore phase

WE ARE ESSENTIAL TO ALBERTA | NAIT

Checkpoint

Students should be ready to show

- a successful login
- a successful restart that restores access
- one invalid-token test that clears the saved session safely
- a logout that clears restored access
- one explanation of why `/auth/me` is part of startup

Status language

- `resolved` : session restores and clears correctly
- `partial` : save or restore works, but the full loop is incomplete
- `blocked` : startup restore or route gating is still broken

WE ARE ESSENTIAL TO ALBERTA | NAIT

Lesson 22 Handoff

Next lesson:

- use this persisted auth flow while making more authenticated requests
- debug protected-request problems with the stored-session path already in place

WE ARE ESSENTIAL TO ALBERTA | NAIT

What We Learned Today

- in-memory auth is not enough for real app sessions
- secure storage keeps the auth session available between launches
- startup restore needs its own temporary state
- `/auth/me` is a clean validation checkpoint for restored sessions
- sign-out must clear stored and live auth data together

WE ARE ESSENTIAL TO ALBERTA | NAIT